



HCM-UTE

TRƯỜNG ĐẠI HỌC

CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH

HCMC University of Technology and Engineering

HỌC PHẦN: TƯƠNG TÁC DỮ LIỆU TRỰC QUAN

Mã học phần: IDVI333677

Giảng viên: Ths. Đoàn Minh Trí

Email liên hệ: doanminhtri@hcmute.edu.vn





CHƯƠNG 4 - THAO TÁC VỚI DỮ LIỆU

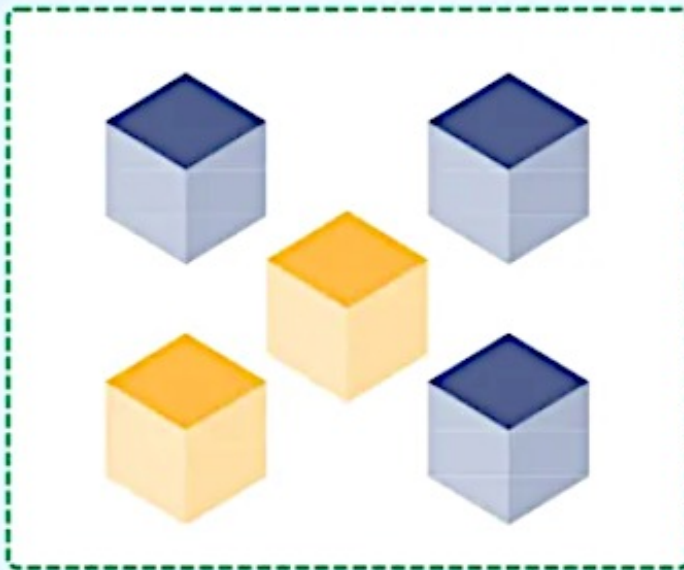
MỤC TIÊU

1. Nắm vững **mức độ chi tiết của dữ liệu** (Granularity / Level of Detail).
2. Thành thạo **Sorting, Filtering và Grouping**.
3. Hiểu và vận dụng được các **thao tác tính tổng hợp** (Aggregation), tính **tổng phụ** (Subtotal) và **phần trăm** (Percentage).
4. Thực hành thao tác dữ liệu và trực quan hóa tương tác bằng Python (pandas, numpy).

NỘI DUNG CHÍNH

1. Mức độ chi tiết của dữ liệu
2. Phân loại và lọc dữ liệu
3. Gom nhóm, tập hợp và thứ bậc
4. Tính toán tổng hợp: Tổng, tổng phụ và phần trăm
5. Thực hành thao tác dữ liệu gắn liền với biểu đồ trực quan

Detailed data



General data

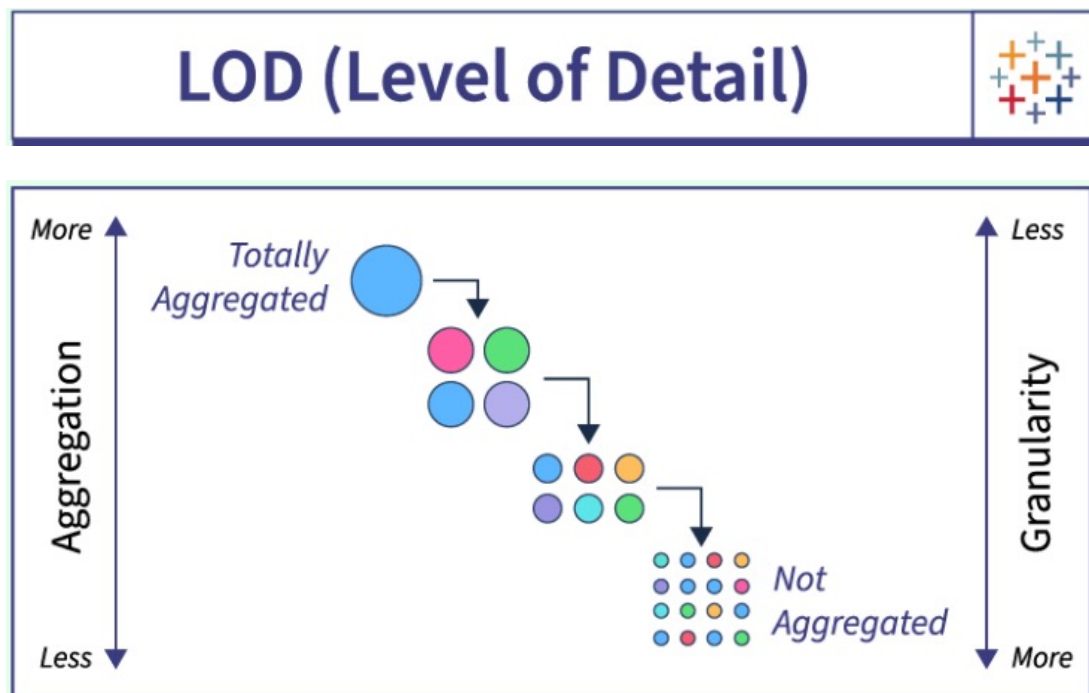


1. MỨC ĐỘ CHI TIẾT CỦA DỮ LIỆU (LEVEL OF DETAIL - LOD)



HCM-UTE

1. MỨC ĐỘ CHI TIẾT CỦA DỮ LIỆU (LEVEL OF DETAIL - LOD)

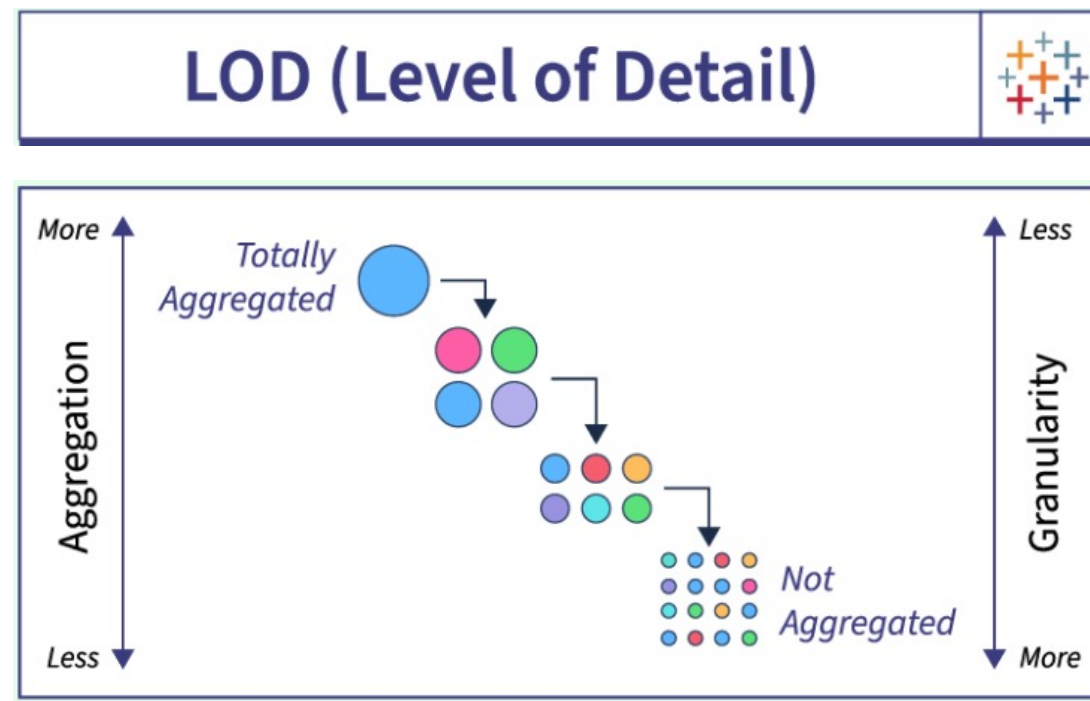


What is LoD?

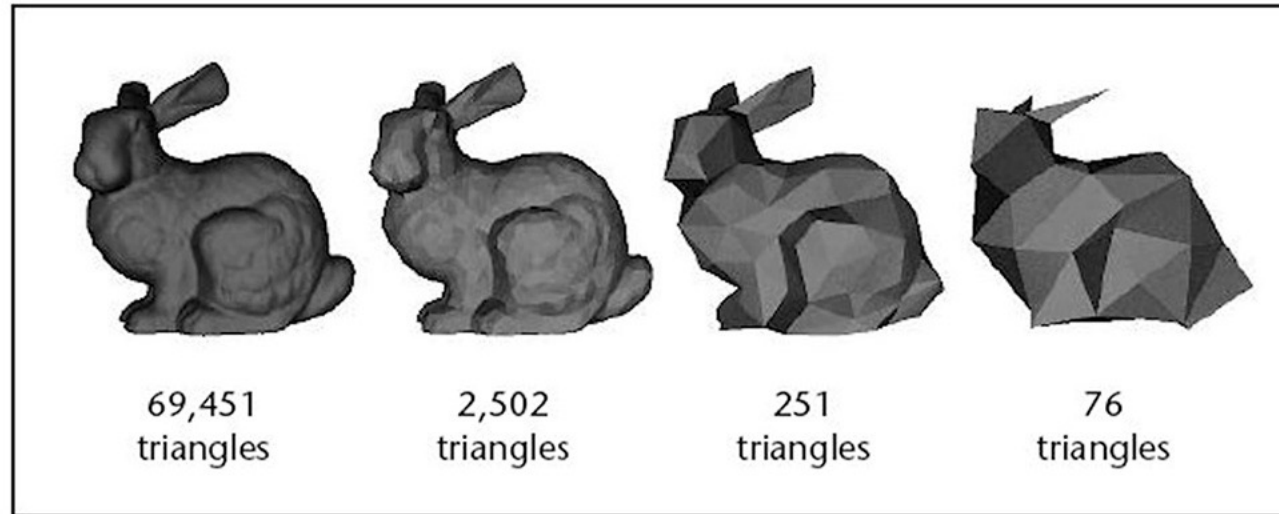
1. MỨC ĐỘ CHI TIẾT CỦA DỮ LIỆU (LEVEL OF DETAIL - LOD)

Khái niệm: Mức độ chi tiết (hoặc độ mịn - Granularity) đại diện cho *cấp độ mà dữ liệu được tổng hợp*.

- High Granularity:** Mức độ chi tiết cao, đại diện cho dữ liệu thô, giao dịch từng phút, thông tin khách hàng lẻ, v.v...
- Low Granularity:** Mức độ chi tiết thấp, thường đại diện cho dữ liệu đã được tổng hợp theo tháng/năm, theo quốc gia, v.v.



1. MỨC ĐỘ CHI TIẾT CỦA DỮ LIỆU (LEVEL OF DETAIL - LOD)



Tại sao quan trọng trong trực quan hóa?

- ✓ Quyết định loại biểu đồ phù hợp.
- ✓ Tránh tình trạng quá tải thông tin



1. MỨC ĐỘ CHI TIẾT CỦA DỮ LIỆU (LEVEL OF DETAIL - LOD)

Date	Category	Sales
2026-01-01	Electronics	250
2026-01-02	Clothing	400
2026-01-03	Electronics	150
<i>(Chứa 100 dòng dữ liệu lẻ tẻ từng ngày)</i>		

1

Date	Category	Sales
2026-01	Clothing	4,200
2026-01	Electronics	5,800
2026-01	Food	3,100
<i>(Đã gộp 100 dòng thô thành các dòng tổng hợp gọn gàng theo Tháng và Ngành hàng)</i>		

2



1. MỨC ĐỘ CHI TIẾT CỦA DỮ LIỆU (LEVEL OF DETAIL - LOD)

1

Ban đầu: Dữ liệu có mức độ chi tiết rất cao (High Granularity) vì mỗi dòng đại diện cho giao dịch của **1 ngày**. Nếu vẽ biểu đồ ngay lập tức với 100 dòng này, biểu đồ sẽ rất rối và chi tiết vụn vặt.

2

Biến đổi từ **Dữ liệu thô vụn vặt** thành **Dữ liệu tổng hợp gọn gàng**, giúp dễ dàng vẽ biểu đồ cột, so sánh doanh thu của 3 ngành hàng qua từng tháng mà không bị ã bởi hàng trăm điểm dữ liệu lẻ



2. PHÂN LOẠI VÀ LỌC DỮ LIỆU (SORTING & FILTERING)



Tạo dữ liệu giả lập sử dụng cho minh họa bài học

```
import pandas as pd
import numpy as np

# Tạo dữ liệu mẫu giao dịch bán hàng
np.random.seed(42)
df = pd.DataFrame({
    'Date': pd.date_range(start='2026-01-01',
                           periods=100, freq='D'),
    'Category': np.random.choice(['Electronics',
                                   'Clothing', 'Food'], 100),
    'Sales': np.random.randint(100, 1000, 100)
})

print("Dữ liệu thô (Chi tiết cao):")
print(df.head(3))
```



2. PHÂN LOẠI VÀ LỌC DỮ LIỆU (SORTING & FILTERING) | SORTING

Mục đích

1. **Tăng tính so sánh:** Giúp người xem lập tức nhận diện giá trị lớn nhất, nhỏ nhất hoặc xu hướng tăng/giảm.
2. **Ứng dụng trực quan:** Sắp xếp lại thứ tự biểu đồ cột (Bar Chart), biểu đồ đường (Line Chart) theo tiêu chí phân tích.

Cú pháp & Phương thức trong Pandas

Phương thức chính: **df.sort_values()**

```
# Sắp xếp DataFrame theo doanh số (Sales) giảm dần  
sorted_df = df.sort_values(by='Sales', ascending=False)
```

Các tham số quan trọng:

- ✓ **by:** Cột (hoặc danh sách các cột) cần sắp xếp.
- ✓ **ascending=True** (Mặc định: Tăng dần) hoặc **ascending=False** (Giảm dần).
- ✓ **inplace=True** (Ghi đè DataFrame cũ) hoặc trả về DataFrame mới.



2. PHÂN LOẠI VÀ LỌC DỮ LIỆU (SORTING & FILTERING) | FILTERING

Khái niệm & Mục đích

1. **Loại bỏ nhiễu:** Cắt bỏ các điểm dữ liệu ngoại lệ (outliers) hoặc không liên quan.
2. **Tập trung phân tích:** Chỉ giữ lại các bản ghi đáp ứng tiêu chí cụ thể để đưa vào biểu đồ.

Các kỹ thuật Lọc phổ biến trong Pandas

- Boolean Indexing (Lọc theo điều kiện Logic)
- Sử dụng `.loc[ ]` (Truy xuất theo Nhãn & Điều kiện)
- Sử dụng `.query()` (Cú pháp chuỗi ngắn gọn)



HCM-UTE

2. PHÂN LOẠI VÀ LỌC DỮ LIỆU (SORTING & FILTERING) | FILTERING

So sánh
Boolean
Indexing và
.loc[]

Tiêu chí	Boolean Indexing (df[...])	Phương thức .loc[] (df.loc[...])
Bản chất	Lọc trực tiếp dựa trên điều kiện True/False	Bộ định vị dựa trên nhãn dòng và nhãn cột (Label Location)
Chọn cột linh hoạt	Khó chọn cột cụ thể trong 1 bước (phải viết nối tiếp <code>df[...]</code> <code>['Sales']</code>)	Chọn dòng và chọn cột rất gọn trong 1 lệnh: <code>df.loc[dòng, cột]</code>
Cú pháp ví dụ	<code>df[df['Sales'] > 500]</code>	<code>df.loc[df['Sales'] > 500, ['Category', 'Sales']]</code>
Sửa đổi dữ liệu	Dễ gặp cảnh báo lỗi <i>SettingWithCopyWarning</i> khi gán giá trị mới	Chuẩn mực trong Pandas, gán giá trị mới an toàn tuyệt đối



2. PHÂN LOẠI VÀ LỌC DỮ LIỆU (SORTING & FILTERING) | FILTERING

Boolean Indexing

```
df[(df['Category'] == 'Electronics') & (df['Sales'] > 1500)]
```

Sử dụng .loc[]

```
df.loc[df['Sales'] > 500, ['Date', 'Sales']]
```

Sử dụng .query()

```
df.query("Category == 'Electronics' and Sales > 500")
```

VÍ VỤ VẬN DỤNG

Sử dụng dữ liệu giả lập, Thực hiện các yêu cầu sau đây:

1. Áp dụng kỹ thuật sắp xếp giảm dần theo Sales
2. Áp dụng 3 kỹ thuật lọc dữ liệu slide 13 theo thông số slide 14.
3. Vẽ biểu đồ Bar Chart hiển thị Top 10 ngày có doanh số cao nhất

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Tạo dữ liệu mẫu giao dịch bán hàng
np.random.seed(42)
df = pd.DataFrame({
    'Date': pd.date_range(start='2026-01-01',
                           periods=100, freq='D'),
    'Category': np.random.choice(['Electronics',
                                   'Clothing', 'Food'], 100),
    'Sales': np.random.randint(100, 1000, 100)
})

print("Dữ liệu thô (Chi tiết cao):")
print(df.head(3))
print("*"*10)
```

VÍ VỤ VẬN DỤNG

Sử dụng dữ liệu giả lập, Thực hiện các yêu cầu sau đây:

1. Áp dụng kỹ thuật sắp xếp giảm dần theo Sales
2. Áp dụng 3 kỹ thuật lọc dữ liệu slide 13 theo thông số slide 14
3. Vẽ biểu đồ Bar Chart hiển thị Top 10 ngày có doanh số cao nhất

```
# Sắp xếp DataFrame theo doanh số (Sales) giảm dần
print(" Sắp xếp giảm dần theo doanh số")
print("*"*10)
sorted_df = df.sort_values(by='Sales', ascending=False)
print("Sorted dataset:")
print(sorted_df)
print("*"*10)
```

VÍ VỤ VẬN DỤNG

Sử dụng dữ liệu giả lập, Thực hiện các yêu cầu sau đây:

1. Áp dụng kỹ thuật sắp xếp giảm dần theo Sales
2. Áp dụng 3 kỹ thuật lọc dữ liệu slide 13 theo thông số slide 14
3. Vẽ biểu đồ Bar Chart hiển thị Top 10 ngày có doanh số cao nhất

```
# Lọc dữ liệu theo các trường hợp
# trường hợp 1: Theo điều kiện logic:
print(" Lọc theo điều kiện")
print("*"*10)
ta = df[(df['Category'] == 'Electronics') & (df['Sales'] > 1500)]
print(ta)
print("*"*10)

# trường hợp 2: Theo phương thức loc:
print(" Lọc theo phương thức loc")
print("*"*10)
tb = df.loc[df['Sales'] > 500, ['Date', 'Sales']]
print(tb)
print("*"*10)

# trường hợp 3: Dùng query:
print(" Lọc theo phương thức dùng query")
print("*"*10)
tc = df.query("Category == 'Electronics' and Sales > 500")
print(tc)
print("*"*10)
```

VÍ VỤ VẬN DỤNG

Sử dụng dữ liệu giả lập, Thực hiện các yêu cầu sau đây:

1. Áp dụng kỹ thuật sắp xếp giảm dần theo Sales
2. Áp dụng 3 kỹ thuật lọc dữ liệu slide 13 theo thông số slide 14
3. Vẽ biểu đồ Bar Chart hiển thị Top 10 ngày có doanh số cao nhất

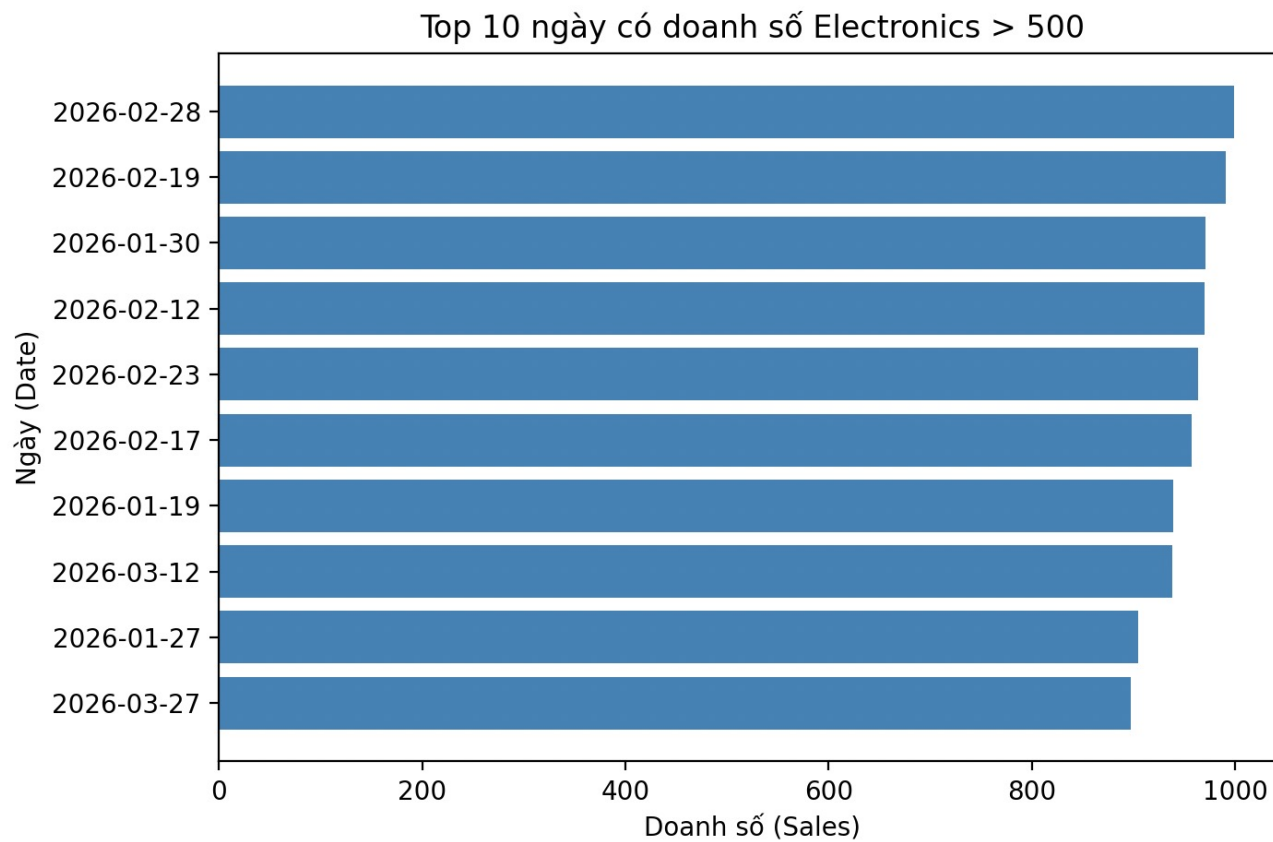
```
#Vẽ biểu đồ cột
top10 = sorted_df.head(10)
y_dates = top10["Date"].dt.strftime('%Y-%m-%d')
plt.barh(y=y_dates, width=top10["Sales"], color='steelblue')
plt.gca().invert_yaxis()

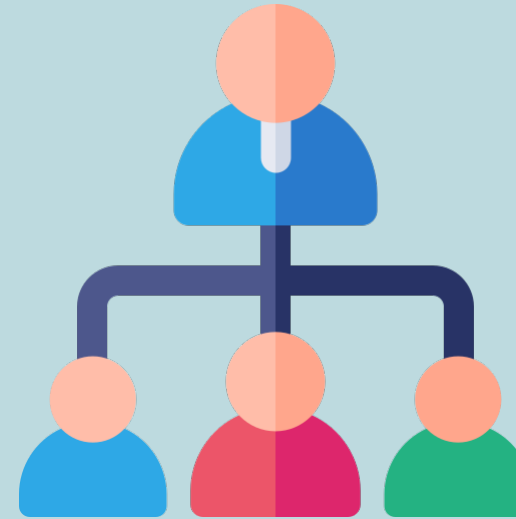
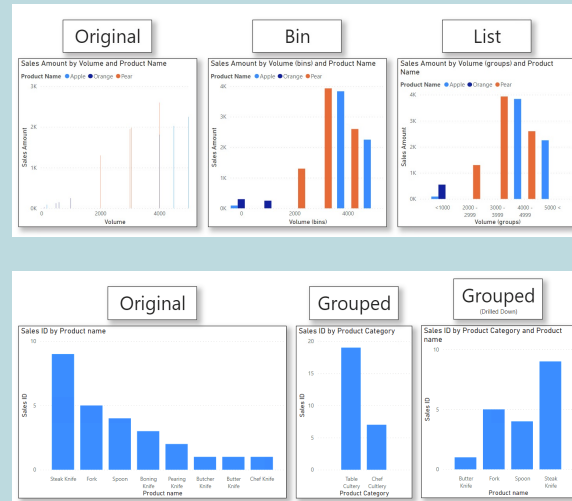
plt.title('Top 10 ngày có doanh số Electronics > 500')
plt.xlabel('Doanh số (Sales)')
plt.ylabel('Ngày (Date)')
plt.tight_layout()
plt.show()
```

VÍ VỤ VẬN DỤNG

Sử dụng dữ liệu giả lập, Thực hiện các yêu cầu sau đây:

1. Áp dụng kỹ thuật sắp xếp giảm dần theo Sales
2. Áp dụng 3 kỹ thuật lọc dữ liệu slide 13 theo thông số slide 14
3. Vẽ biểu đồ Bar Chart hiển thị Top 10 ngày có doanh số cao nhất





3. NHÓM, BỘ VÀ BẬC (GROUPS, SETS & BINNING/HIERARCHY)



3. NHÓM, BỘ VÀ BẬC (GROUPS, SETS & BINNING/HIERARCHY)

3.1. GOM NHÓM DỮ LIỆU (GROUPING)

3.1.1. Khái niệm & Mục đích:

Gom nhóm: Chuyển đổi dữ liệu thô (Raw Data) thành **các nhóm có chung thuộc tính** để thực hiện so sánh hoặc tính toán tổng hợp.

Mô hình Split - Apply - Combine trong Pandas:

- *Split (Tách):* Chia dữ liệu thành các nhóm dựa trên tiêu chí chỉ định.
- *Apply (Áp dụng):* Tính toán chỉ số cho từng nhóm (Ví dụ: `sum()`, `mean()`, `count()`).
- *Combine (Kết hợp):* Gộp các kết quả lại thành một bảng dữ liệu mới.

3.1.2. Cú pháp cơ bản:

Gom nhóm theo cột Category và tính điểm trung bình cột Sales:

```
df.groupby('Category')['Sales'].mean()
```

Cái gì?

cột nào ?

Hàm nào ?



3. NHÓM, BỘ VÀ BẬC (GROUPS, SETS & BINNING/HIERARCHY)

3.1. GOM NHÓM DỮ LIỆU (GROUPING)

```
import pandas as pd
```

```
# 1. Tạo bảng dữ liệu mẫu
```

```
data = {  
    'Sản phẩm': ['Táo', 'Táo', 'Cam', 'Cam', 'Chuối'],  
    'Số lượng': [10, 15, 8, 12, 20]  
}
```

```
df = pd.DataFrame(data)
```

```
# 2. Gom nhóm theo 'Sản phẩm' và tính tổng 'Số lượng'
```

```
tong_so_luong = df.groupby('Sản phẩm')['Số lượng'].sum()
```

```
print("--- Bảng dữ liệu thô ---")
```

```
print(df)
```

```
print("\n--- Kết quả gom nhóm (Tổng số lượng) ---")
```

```
print(tong_so_luong)
```

--- Bảng dữ liệu thô ---

	Sản phẩm	Số lượng
0	Táo	10
1	Táo	15
2	Cam	8
3	Cam	12
4	Chuối	20

--- Kết quả gom nhóm (Tổng số lượng) ---

Sản phẩm	
Cam	20
Chuối	20
Táo	25



GIẢI LAO 15 PHÚT



3. NHÓM, BỘ VÀ BẬC (GROUPS, SETS & BINNING/HIERARCHY)

3.2. TẬP HỢP VÀ THỨ BẬC (SETS & HIERARCHY)

3.2.1. Tập hợp

Định nghĩa: Nhóm các đối tượng thỏa mãn một tập điều kiện logic phục vụ mục đích so sánh với phần còn lại.

Ứng dụng: Xác định nhóm khách hàng VIP, danh sách mặt hàng bán chạy (Top N).

3.2.2. Phân cấp / Thứ bậc

Định nghĩa: Dữ liệu có tính chất bao hàm theo nhiều cấp độ từ rộng đến hẹp.

Ví dụ: Quốc gia → Tỉnh / Thành phố → Quận / Huyện.

Ứng dụng trong Trực quan hóa:

Cho phép người dùng thực hiện thao tác **Drill-down** (đi sâu vào chi tiết) hoặc **Roll-up** (tổng hợp lên cấp cao hơn) trên biểu đồ.

Ví dụ về Tập hợp

Mục đích: Tạo logic điều kiện để phân loại tập hợp đối tượng (Ví dụ: Lọc nhóm **Khách hàng VIP** đáp ứng tiêu chí tổng chi tiêu > 1,000 USD hoặc **Top N** sản phẩm bán chạy)

Tập hợp mã khách hàng VIP: {'C01'}

Bảng dữ liệu phân loại Tập hợp:

	Customer_ID	Total_Spend	Is_VIP
0	C01	1200	True
1	C02	450	False
2	C03	100	False
3	C04	800	False
4	C05	500	False
5	C06	300	False

```
import pandas as pd
```

```
# Tạo dữ liệu giả lập về Khách hàng và Doanh số
```

```
df_sales = pd.DataFrame({  
    'Customer_ID': ['C01', 'C02', 'C03', 'C04', 'C05', 'C06'],  
    'Total_Spend': [1200, 450, 100, 800, 500, 300]  
})
```

```
# 1. Tạo Tập hợp (Set) – Khách hàng VIP (Total_Spend > 1000)
```

```
vip_customers = set(df_sales[df_sales['Total_Spend'] > 1000]['Customer_ID'])
```

```
# 2. Tạo cột Phân loại Set trong DataFrame
```

```
df_sales['Is_VIP'] = df_sales['Customer_ID'].isin(vip_customers)
```

```
print("Tập hợp mã khách hàng VIP:", vip_customers)
```

```
print("\nBảng dữ liệu phân loại Tập hợp:")
```

```
print(df_sales)
```

**HCM-UTE**

Ví dụ về Phân cấp / Thứ bậc (Hierarchy)

Mục đích: Multi-level Grouping theo các cấp độ bao hàm (Ví dụ: Region → City), cho phép tính toán tổng hợp (Roll-up) và sẵn sàng cho thao tác đào sâu dữ liệu (Drill-down).

Dữ liệu được tổ chức theo Thứ bậc (Hierarchy):

Region	City	
Miền Bắc	Hà Nội	500
	Hải Phòng	200
Miền Nam	Cần Thơ	300
	TP.HCM	800

Name: Sales, dtype: int64

Kết quả Roll-up (Cấp Region):

Region	
Miền Bắc	700
Miền Nam	1100

Name: Sales, dtype: int64

```
import pandas as pd

# Tạo dữ liệu doanh số theo Phân cấp Địa lý (Region -> City)
df_geo = pd.DataFrame({
    'Region': ['Miền Bắc', 'Miền Bắc', 'Miền Nam', 'Miền Nam'],
    'City': ['Hà Nội', 'Hải Phòng', 'TP.HCM', 'Cần Thơ'],
    'Sales': [500, 200, 800, 300]
})

# Gom nhóm theo Thứ bậc (Hierarchy Grouping: Cấp 1 -> Cấp 2)
hierarchy_sales = df_geo.groupby(['Region', 'City'])['Sales'].sum()

print("Dữ liệu được tổ chức theo Thứ bậc (Hierarchy):")
print(hierarchy_sales)

# Thao tác Roll-up (Tổng hợp ngược lên Cấp 1 - Region)
rollup_sales = df_geo.groupby('Region')['Sales'].sum()

print("\nKết quả Roll-up (Cấp Region):")
print(rollup_sales)
```



3. NHÓM, BỘ VÀ BẬC (GROUPS, SETS & BINNING/HIERARCHY)

3.3. PHÂN KHOẢNG DỮ LIỆU (BINNING / DISCRETIZATION)

3.3.1. Khái niệm

Binning: Kỹ thuật chuyển đổi các biến số liên tục (Continuous variables) thành các nhóm phân loại cố định (Categorical bins).

Ví dụ: Chuyển cột Age (15, 25, 45, 65...) thành các khoảng nhãn (Thiếu niên, Thanh niên, Trung niên, Người cao tuổi).

3.3.2. Hai phương pháp Binning chính trong Pandas

Phương pháp	Hàm	Đặc điểm
Khoảng cố định	pd.cut()	Chia khoảng dựa trên giá trị bờ quy định trước (Width bằng nhau hoặc tùy chỉnh).
Tần suất bằng nhau	pd.qcut()	Chia khoảng dựa trên tỷ lệ phần trăm (Quantiles) sao cho số lượng mẫu ở mỗi nhóm bằng nhau.



Ví dụ vận dụng: phân khoảng dữ liệu & trực quan hóa

Yêu cầu: Sử dụng ngôn ngữ Python và các thư viện pandas, matplotlib để hoàn thành các nhiệm vụ sau:

1. Khởi tạo dữ liệu: Tạo một DataFrame chứa danh sách độ tuổi của 10 khách hàng:

`Ages = [15,22,25,35,42,58,65,71,18,30]`

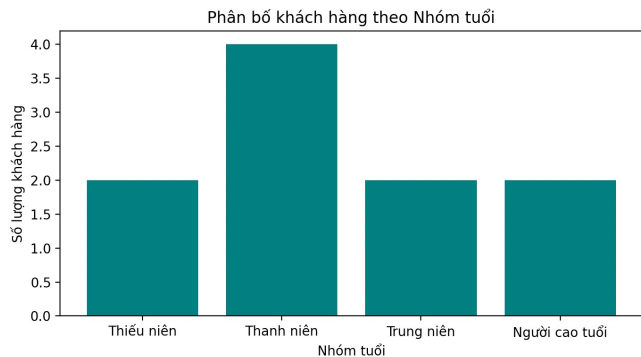
2. Phân khoảng (Binning): Sử dụng hàm `pd.cut()` để chia độ tuổi thành 4 nhóm theo các mốc bờ `[0, 18, 40, 60, 100]` tương ứng với các nhãn:

1. 'Thiếu niên' (0 – 18]
2. 'Thanh niên' (18 – 40]
3. 'Trung niên' (40 – 60]
4. 'Người cao tuổi' (60 – 100]

3. Thống kê: Đếm số lượng khách hàng thuộc mỗi nhóm tuổi vừa phân chia.

4. Trực quan hóa: Vẽ **biểu đồ cột (Bar chart)** hiển thị số lượng khách hàng theo từng nhóm tuổi. Đặt màu cột thành màu xanh teal ('teal'), thêm tiêu đề và nhãn cho các trục tọa độ.

Ví dụ vận dụng: phân khoảng dữ liệu & trực quan hóa



```
import pandas as pd
import matplotlib.pyplot as plt
```

1. Tạo dữ liệu

```
df_customers = pd.DataFrame({'Age': [15, 22, 25, 35, 42, 58, 65, 71, 18, 30]})
```

2. Chia khoảng độ tuổi

```
bins = [0, 18, 40, 60, 100]
```

```
labels = ['Thiếu niên', 'Thanh niên', 'Trung niên', 'Người cao tuổi']
```

```
df_customers['Age_Group'] = pd.cut(df_customers['Age'], bins=bins, labels=labels)
```

3. Đếm số lượng theo nhóm

```
group_counts = df_customers['Age_Group'].value_counts().sort_index()
```

4. Vẽ biểu đồ

```
plt.figure(figsize=(7, 4))
```

```
plt.bar(group_counts.index.astype(str), group_counts.values, color='teal')
```

```
plt.title('Phân bố khách hàng theo Nhóm tuổi')
```

```
plt.xlabel('Nhóm tuổi')
```

```
plt.ylabel('Số lượng khách hàng')
```

```
plt.tight_layout()
```

```
plt.show()
```



4. TỔNG VÀ TỔNG PHỤ (SUMS & SUBTOTALS)

4. TỔNG VÀ TỔNG PHỤ (GRAND TOTAL & SUBTOTALS)

4.1. Khái niệm & Khác biệt

Grand Total (Tổng toàn bộ): Con số tổng duy nhất đại diện cho toàn bộ tập dữ liệu (Ví dụ: Tổng doanh thu toàn công ty).

Subtotals (Tổng phụ): Con số tổng aggregated theo từng nhóm/phân đoạn cụ thể (Ví dụ: Tổng doanh thu theo từng tháng, từng ngành hàng).



4. TỔNG VÀ TỔNG PHỤ (GRAND TOTAL & SUBTOTALS)

4.2. CÁC PHƯƠNG PHÁP TÍNH SUBTOTALS TRONG PANDAS

4.2.1. Phương pháp 1: Kết hợp `groupby()` và `agg()`

Áp dụng khi cần gom nhóm theo danh sách thuộc tính và tính tổng phụ cho từng nhóm.

```
# Tính Subtotal theo Category
```

```
df.groupby('Category')['Sales'].agg('sum')
```

4.2.2. Phương pháp 2: Sử dụng `pivot_table()` với `margins=True`

Giúp tạo bảng tổng hợp 2 chiều (Cross-tabulation).

Chú ý: Tham số `margins=True` tự động tính:

- **Cột Tổng phụ (Subtotal theo Hàng):** Tổng theo từng Category qua các tháng.
- **Hàng Tổng phụ (Subtotal theo Cột):** Tổng của tất cả Category trong từng tháng.
- **Grand Total:** Ô góc dưới cùng bên phải.

4. TỔNG VÀ TỔNG PHỤ (GRAND TOTAL & SUBTOTALS)

4.2. CÁC PHƯƠNG PHÁP TÍNH SUBTOTALS TRONG PANDAS

4.2.2. Phương pháp 2: Sử dụng `pivot_table()` với `margins=True`

```
datausingpvt= pd.pivot_table(  
    data,  
    values=None,  
    index=None,  
    columns=None,  
    aggfunc='mean',  
    margins=True,  
    margins_name='All'  
)
```

Trong đó:

1. **data**: Tên DataFrame nguồn chứa dữ liệu cần tổng hợp.
2. **values**: Cột dữ liệu số học muốn đưa vào tính toán (ví dụ: `Doanh_Thu`, `Sales`).
3. **index**: Trường dữ liệu được gom nhóm và hiển thị theo chiều dọc (dừng lại ở các hàng).
4. **columns**: Trường dữ liệu được tách nhóm và trải rộng ra theo chiều ngang (các cột).
5. **aggfunc**: Hàm thống kê áp dụng lên dữ liệu (mặc định là tính trung bình `'mean'`, có thể đổi thành `'sum'`, `'count'`, `'min'`, `'max'`).
6. **margins**: Nhận giá trị `True` để bật tính năng tự động tính toán tổng phụ (*Subtotals*) và tổng toàn bộ (*Grand Total*).
7. **margins_name**: Tùy chỉnh tên hiển thị nhãn cho hàng và cột tổng (ví dụ: `'Tong_Cong'`, `'Total'`).

Ví dụ vận dụng: Sử dụng PIVOT TABLE TÍNH SUBTOTALS

Cho đoạn mã khởi tạo data trái cây như sau:

```
import pandas as pd

# Tạo dữ liệu bán trái cây tại các cửa hàng
data = {
    'Cua_Hang': ['Chi nhánh A', 'Chi nhánh A', 'Chi nhánh B', 'Chi nhánh B', 'Chi nhánh A', 'Chi nhánh B'],
    'Loai_Trai_Cay': ['Táo', 'Cam', 'Táo', 'Cam', 'Táo', 'Táo'],
    'Doanh_Thu': [150, 200, 180, 220, 130, 170]
}
df_fruits = pd.DataFrame(data)
```

Yêu cầu: viết mã nguồn, sử dụng Pivot Table tính Subtotals & Grand Total để có được bản thống kê kết quả:

Loai_Trai_Cay	Cam	Táo	Tong_Cong	
Cua_Hang				
Chi nhánh A	200	280	480	← Tổng doanh thu Chi nhánh A (Subtotal)
Chi nhánh B	220	350	570	← Tổng doanh thu Chi nhánh B (Subtotal)
Tong_Cong	420	630	1050	← Grand Total (Góc dưới cùng bên phải)
	▲	▲		
	└──┬──┘			Tổng doanh thu theo từng Loại trái cây (Subtotal)



5. TÍNH TỶ LỆ PHẦN TRĂM (PERCENTAGES)



5. TÍNH TỶ LỆ PHẦN TRĂM (PERCENTAGES)

Vai trò: Trực quan hóa dưới dạng tỷ lệ đóng góp (Percentage of Total, Cumulative Percentage) giúp góc nhìn trực quan sâu sắc hơn số tuyệt đối.

Các dạng phần trăm thường gặp:

1. % trên Tổng số toàn bộ dataset (% of Total).
2. % trong nội bộ từng nhóm (% of Group).



Ví dụ vận dụng: TÍNH % ĐÓNG GÓP ĐIỂM SỐ MÔN HỌC

Giả xử môn học *Tương tác dữ liệu trực quan* có cấu trúc điểm thành phần và trọng số % như sau:

- ✓ **Chuyên cần: 10%**
- ✓ **Bài tập / Mini Lab: 20%**
- ✓ **Kiểm tra Giữa kỳ: 20%**
- ✓ **Đề tài Môn học (Đồ án): 50%**

Tổng điểm môn học của sinh viên: 8.60 / 10.0

Bảng chi tiết % đóng góp điểm số:

	Thanh_Phan	Diem_So	Diem_Quy_Doi	Percent_Contribution
0	Chuyên cần	10.0	1.0	11.627907
1	Bài tập/MiniLab	8.5	1.7	19.767442
2	Giữa kỳ	7.0	1.4	16.279070
3	Đồ án Môn học	9.0	4.5	52.325581

Yêu cầu: Tính điểm quy đổi đóng góp của từng cột điểm vào **Tổng điểm tổng kết (theo mẫu)** và vẽ biểu đồ tròn thể hiện cấu trúc điểm đóng góp của một sinh viên.



BÀI TẬP VẬN DỤNG



Bài tập 1:

Yêu cầu:

Tạo sinh dữ liệu danh sách giao dịch gồm các cột: Order_ID, Category, Sales, Profit bằng python hoặc GenAI. Trên data này, thực hiện các yêu cầu sau:

1. Lọc ra các đơn hàng thuộc nhóm ngành có Sales > 300.
2. Sắp xếp kết quả thu được theo cột Profit giảm dần.
3. Lấy ra Top 5 đơn hàng có lợi nhuận cao nhất và vẽ biểu đồ cột (Bar chart) thể hiện Profit của 5 đơn hàng này.
4. Lưu notebook: *C4_hovaten_mssv_b1.ipynb*



Bài tập 2:

Đề bài:

Cho dữ liệu độ tuổi của 20 khách hàng: `ages = [18, 22, 25, 27, 31, 35, 40, 41, 45, 52, 55, 60, 62, 65, 19, 29, 38, 48, 51, 70]`.

Yêu cầu:

1. Chia độ tuổi thành 3 nhóm bậc bằng `pd.cut`: 'Trẻ' (17-30], 'Trung niên' (30-50], 'Lớn tuổi' (50-100].
2. Đếm số lượng khách hàng trong từng nhóm.
3. Trực quan hóa số lượng khách hàng theo từng nhóm bằng biểu đồ tròn (Pie chart).
4. Lưu notebook: `C4_hovaten_mssv_b2.ipynb`



Bài tập 3:

Đề bài:

Tạo sinh 1 DataFrame doanh số bán hàng theo khu vực (Region) và doanh thu (Sales) bao gồm 123 sample.

Yêu cầu:

1. Tính tổng Sales theo từng Region bằng hàm groupby().
2. Tạo thêm cột Sales_Percent thể hiện phần trăm đóng góp doanh thu của từng khu vực so với tổng doanh thu toàn bộ.
3. Hiển thị bảng kết quả và vẽ biểu đồ cột đứng thể hiện cột Sales_Percent.
4. Lưu notebook: *C4_hovaten_mssv_b3.ipynb*

MINI PROJECT 1 (10 PHÚT)

Yêu cầu:

Sử dụng dataset `tips` có sẵn trong thư viện Seaborn:

1. Lọc các hóa đơn vào ngày Thứ Bảy (`day == 'Sat'`).
2. Chia khoảng tiền hóa đơn (`total_bill`) thành 3 mức: Thấp, Trung bình, Cao.
3. Tính tổng tiền Tip (`tip`) và phần trăm đóng góp tiền Tip theo từng mức hóa đơn.
4. Trực quan hóa kết quả lên biểu đồ cột (Bar chart).
5. Lưu notebook: *C4_hovaten_mssv_miniproject1.ipynb*



MINI PROJECT 2 (20 PHÚT)

Đề bài: Phân tích & Trực quan hóa hành vi chi tiêu (Dataset: tips)

Bối cảnh: Sử dụng dataset thô tips (tích hợp sẵn trong thư viện seaborn) chứa thông tin tiền hóa đơn, tiền tip, giới tính, ngày trong tuần, v.v...

Yêu cầu chi tiết:

- ✓ **Bước 1 (Lọc & Sắp xếp):** Lọc ra các hóa đơn xảy ra vào buổi tối (time == 'Dinner'). Sau đó sắp xếp theo total_bill giảm dần.
- ✓ **Bước 2 (Gom nhóm & Tổng phụ - Subtotals):** Gom nhóm theo day (Ngày trong tuần) và sex (Giới tính khách trả tiền), tính tổng tiền tip nhận được.
- ✓ **Bước 3 (Phân khoảng - Binning):** Tạo một cột mới bill_group chia total_bill thành 3 mức: 'Thấp', 'Trung bình', 'Cao'.
- ✓ **Bước 4 (Trực quan hóa):** Vẽ biểu đồ cột chồng (Stacked Bar Chart) hoặc biểu đồ cột nhóm thể hiện phần trăm đóng góp tiền tip theo từng mức hóa đơn (bill_group) giữa Nam và Nữ.
- ✓ **Bước 5: Lưu notebook: *C4_hovaten_mssv_miniproject2.ipynb***



THANK YOU !